

# 把初始 Token Embedding 加回每一层（验证损失 -0.22） - Transformer（LLM）预训练

你正在预训练一个 LLM。

你会发现，token 穿过越来越深的 Transformer 层时，会获得越来越多上下文。

但等等 — 在处理这些 token 的过程中，我们是不是也把 token 自己最原始的信息埋到了新的上下文下面？

如果只使用模型已经算出来的东西，你会怎么把这份原始信息交还给每一层，看看它是否有帮助？

提示：原始 embedding 还在模型底部 — layer 0 之后就没有再被使用。

## 我的实验

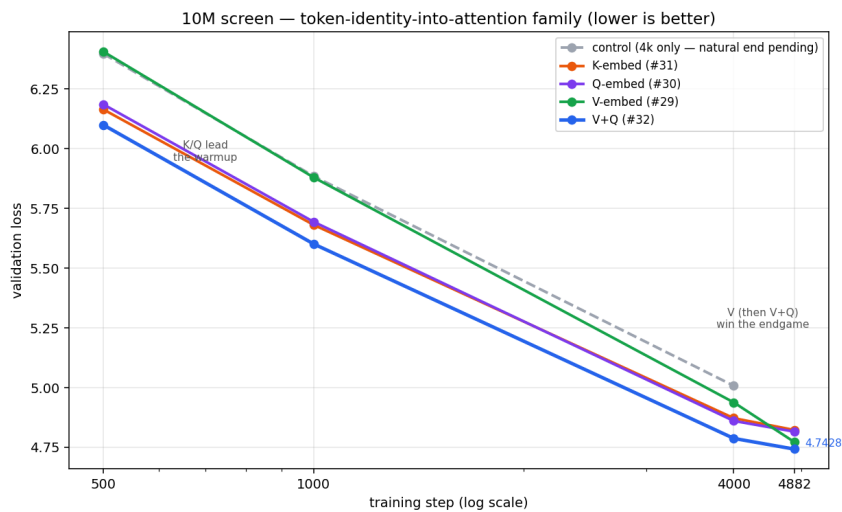
10M 参数 Transformer，在约 20M token 上训练。

token 的原始 embedding 在 layer 0 计算出来，之后就不再使用 — 上下文会随着深度稀释它。所以我试着在每一层 attention 里把它加回去（query、key 或 value）。

## 结果（单 seed @ 4,882 steps）

| 注入位置     | 额外参数  | 验证损失   |
|----------|-------|--------|
| K（key）   | +0.7% | 4.8228 |
| Q（query） | +2.2% | 4.8159 |
| V（value） | +0.7% | 4.7728 |
| V+Q      | +2.9% | 4.7428 |

重点不是排名，而是曲线形状：K/Q 早期领先，V 后期胜出，V+Q 全程最好。



## 诚实的限制

- 只有一个 seed。run-to-run 方差约 0.06 – 0.16，所以 V 和 Q 的差距只能说明方向，不是最终结论。
- 这是一个 20M-token screen，不是完整记录。
- 自然结束的 control 还没跑完 — 目前还没有 4,882 step 上与 control 的精确差值。

## 我是怎么加回去的

每个位置只加一行 — 一个从全零开始的小矩阵：

$$V = W_v \cdot h + \underbrace{W_{ve} \cdot e}_{\text{W\_ve 从 0 开始} \rightarrow \text{step 0 与 baseline bit-for-bit 相同}} \quad \# e = \text{layer 0 的原始 token embedding}$$

为什么从 0 开始：

- Step 0 = 精确 baseline —  $W_{ve} \cdot e = 0$ ，所以之后的任何提升都来自这个机制，而不是某次幸运的重新初始化。
- 自己挣到开启资格 — 零权重依然会得到非零梯度，所以它从 step 1 就开始训练，但只有在降低 loss 时才会长大。
- 下行风险很小 — 最坏情况是它保持在约 0 (no-op)；你不需要冒着破坏 baseline 的风险来测试这个想法。

同一行也适用于 Q 和 K — 这就是我在测试的「加到哪里」。

这些矩阵：每一层都有自己的矩阵（24 层 → 24 个独立矩阵，不共享）。每个矩阵把  $e$ （48 维）映射到对应的 attention slot：

- V / K：每层  $48 \times 48 \rightarrow$  总共约 55k 参数 (+0.7%)
- Q：每层  $144 \times 48 \rightarrow$  总共约 166k 参数 (+2.2%) — 更大是因为 query slot 是完整的  $d_{\text{model}}=144$ ，而 K/V slot 是 48。